[Bitcoin Pricing Dashboard]

Cloud Data Centre Module — Project Report

Author(s)	Jad Jason Fuhr
Student ID(s)	C00290965
Module	Cloud Data Centre
Submission Date	Friday 1st May, 17:00
GitHub Repo	https://github.com/JadFuhr/cloud-dashboard
Word Count	15 pages worth

Abstract

This project demonstrates a cloud hosted static web application that retrieves real time Bitcoin price data using Microsoft Azures serverless deployment model. This application queries the Binance public API to obtain daily Bitcoin (BTC/USDT) candlestick data and provides users with an interactive dashboard that displays the highest, lowest and average Bitcoin prices over a selectable 30-day and 60-day period. This is a fully client-side application using HTML, CSS and JavaScript, with the use of Plotly.js (used for the visualization of this data.
Azure Static Web Apps is used as the hosting platform which ensures this can be globally distributed content, lightweight in terms of its deployment pipeline and automated through GitHub actions

The aim of my work is to show that a cloud platform such as Azure can support applications that are scalable and low maintenance. This report will outline my motivations, performance considerations, system architecture, and deployment workflow. The results show that a serverless static hosting model is very much sufficient for data driven dashboards that don't require backend computation.

Keywords – Cloud Computing, Serverless, Azure Static Web Apps, Data Visualization, Crypto Currency, BTC – Bitcoin, Crypto Currency Analytics, Binance, Plotly, CDN, API, GitHub, VM, OS, Container, REST

I. INTRODUCTION

A. Motivation

Cryptocurrency and the crypto space in general is something that I would like to believe I am fairly well-versed in. Part of my employment background over the summer of 2023 was for a South African startup company under the name QuadX that dealt with Crypto trading, crypto currency and an Artificial Intelligence the team was attempting to implement that would make the decisions behind certain trades based off of variables, a simple example being how the trading chart would move. I also took part in day trading as a hobby during this part of my time in South Africa…That being said, Crypto markets are highly volatile and quick, so real-time access to price information is essential.

Traditional desktop applications require a lot of manual setups ongoing maintenance and are hosted locally. Deploying such tools to the cloud could possibly remove these limitations by providing automatic scaling, always available access and seamless updates

B. Background

Keywords to understand:

- **CPU**: Central Processing Unit, the brain of the computer responsible for executing instructions
- **Memory**: a Vital component that temporarily stores data for quick access by you computers processor
- **BTC**: Bitcoin
- **REST**: Reprisentational State Transfer, is an architectural style for designing networked applications
- **API**: Application Programming Interface
- **Binance**: The world's largest cryptocurrency exchange
- **Plotly**: a high-level charting library
- **CDN**: Content Delivery Network
- **CI/CD**: Continuous integration and Continuous Delivery**,** this is a DevOps practice that automates the building, testing and deployment of software
- **GitHub**: a Web based, cloud hosted platform that provides a centralized space where developers can manage, store and share their code via Git version control
- **OS**: Operating system (e.g.: Windows)
- **VM:** Virtual Machine, a software-based emulation of a physical computer that runs an operating system and applications independently, sharing resources from a singular host
- **Container:** standalone package of software that includes everything you need to run an application

Cloud Computing lets you run applications on someone else's infrastructure instead of your own hardware, this means you get the ability to scale up when needed, ensuring better reliability and a lot less maintenance for you to worry about.
Modern platforms in terms of cloud computing give you a good few ways to deploy, for example, a VM, container or you could go serverless. Platforms like Azure take care of all the underlying infrastructure for you so you aren't going to fuss over CPUs, memory or an OS. You can just deploy your code, and the platform handles the rest.
My project leans into this nicely as everything runs in the browser and there is no backend involved.

The app pulls BTC data directly from Binance REST API and visualizes this information using Plotly. My app loads quickly because of Azures CDN and GitHub actions take care of automatic deployment whenever changes are pushed. These two together are the backbone of how my app is built and shipped.

C. Contributions

Contribution 1: Designed and implemented a could hosted bitcoin dashboard:

- I implemented a fully client-side web app that pulls real time BTC price data from the Binance public API and visualizes this data using Plotly.js.
- This system takes key statistics (High, Low, Average), computes them and supports a selection window (laid out as buttons) for the data pool you wish to look at (30, 60, 90 days)

Contribution 2: Deployed a serverless architecture on Microsoft Azure:

- My application is fully deployed using the Static Web Apps feature on Azure.
- This is a serverless hosting model that has automated updates via GitHub actions and demonstrates how efficiently lightweight analytics tools can be hosted.

Contribution 3: Performance and Comparative Evaluation:

- To track performance, I added code to record API latency, chart rendering times and total load times for different dataset sizes. This was evaluated 30, 60 and 90 day workloads to compare the performance.

D. Report Structure

Section II:

Presents the literature review, summarizing existing research on cloud hosted dashboards, API driven data systems and client side performance measurement.

Section III:

Describes the system architecture, including the design, interactions between components and an architecture diagram

Section IV:

Outlines the experiment setup and performance evaluation, detailing the deployment, metrics used and measured results

Section V:

Provides a discussion regarding the findings, an interpretation of my results, the limitations present and challenges ive encountered

Section VI:

Conclusion to my report, summarizing the project and my possible future work implementations

Section VII:

Screencast Link

II. LITERATURE REVIEW

Modern cloud hosted dashboards rely on a combination of web technologies, asynchronous client side programming, REST based data retrieval, and visualisation frameworks. This section reviews the existing literature and documentation relevant to the technologies used in this project, organised thematically to reflect my architectural components described in Section III.

Web Technologies for Cloud

I would go so far as to say that HTML, CSS, and JavaScript would be the backbone of modern web apps. HTML provides the layout and structure (skeleton) of a webpage by creating elements like headings, paragraphs, and containers. CSS is responsible for presentation, styling, and overall look of a web page and JavaScript adds the interactive dynamic behaviour, allowing the client side logic to provide updates to the webpage. These are extensively documented in MDN Web Docs, which provide guidance with regards to standards maintained by Mozilla and major browser contributors. They are well suited for serverless cloud deployments such as Azure Static Web Apps, where static assets are deployed via a global CDN. [1]-[3],[16]-[18]

Asynchronous JS and API communication

Asynchronous programming is definitely a key part of how modern web applications (hosted or not) interact with services accessed externally. In JavaScript, the fetch() API provides a simple way to make requests without blocking or slowing down my webpage, which means the browser can keep responding while it waits for data to come back and be parsed. MDN's documentation explains how fetch() handles the full request to response cycle, including JSON parsing and error handling [4][15]. The introduction of the async/await() functions makes this even easier to work with by letting devs write code in a style that looks like regular logic. This gets rid of any unwanted complexity in your codebase and makes the code much clearer to read. These features are especially important in this project because the dashboard needs to pull data from the

Binance API without slowing down my interface. Using these async functions makes sure that my webpage stays responsive while all the network requesting is happening in the background.

REST APIs and Cloud based Data Retrieval

REST is one of the most common ways to build web based data services, and the majority of modern APIs are built around it. The idea is straightforward, data is exposed through HTTP endpoints, each request is self contained, and responses typically come back as JSON file. JSON works particularly well here because it works with the JavaScript I used in my project and JS in general, this makes it work well with the values you actually need.

The Binance API [5] follows this same pattern to an extent as it offers public endpoints that return crypto market data so therefore working with the responses in JavaScript feels natural because of how cleanly JSON maps to the kind of data you're already dealing with.

One thing that is worth mentioning is that REST APIs are stateless which means that each request is handled on its own. And because of that, the response time can vary…which isn't anything out of the ordinary since it usually comes down to network conditions, caching, or server load. The small jumps in latency I saw during testing fit exactly with this behaviour.

Client side data visualization

In my opinion, interactive charts are a big part of most dashboards purely with regards the user experience of it all especially when it comes to looking at data. Plotly.js is a good fit for this since it runs in the browser and supports things like responsive layouts without needing any backend work. [6]

Keeping the visualisation on the client side has its benefits as it may reduce the load on the server, removes the need for a backend rendering pipeline, and makes it easy to update charts instantly as new data arrives when pulled and parsed from my API. One of the only downsides I think is worth mentioning with regards to having everything rely on the client is that the user's device has to do the work, so performance depends on their browser and hardware. In my case, Plotly.js handled the Bitcoin price data without any noticeable delay, so it worked well for what the dashboard needed

Summary

The technologies discussed here… HTML, CSS, JavaScript, REST APIs, and client side visualisation libraries are not just widely used individually, together, they form a strong foundation for building dashboards that are lightweight, easy to deploy, and simple to maintain. The documentation and standards behind each technology support the design choices made in this project, and they directly influence the implementation and performance evaluation presented in the following sections.

III. SYSTEM ARCHITECTURE

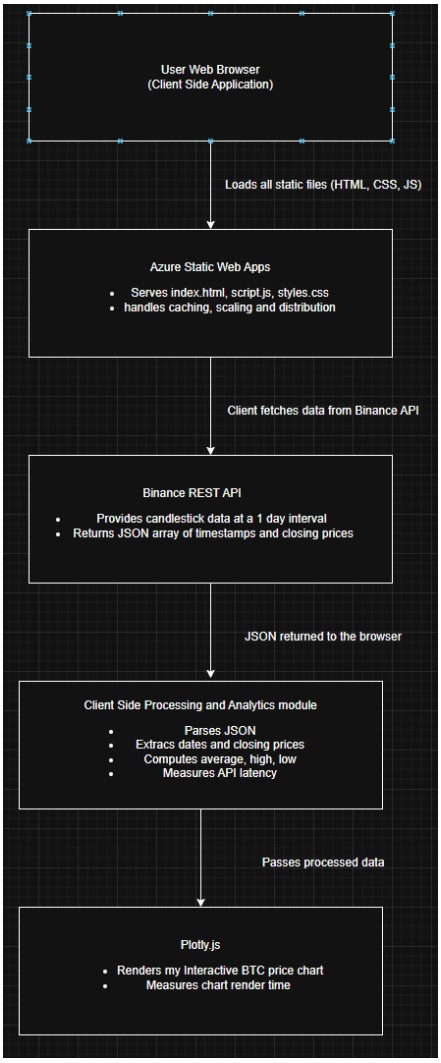
A. High-Level Design

A fully client side system/ serverless web app deployed on Microsoft Azure Static Web Apps. It contains three major components within its architecture:

1. A data retrieving function that gets BTC price data from the Binance REST API.
2. A client side analytic function that computes summary statistics
3. A data visualization function that renders an interactive line graph using Plotly.js

These components interact in sequence to one another. The API retrieves the data, the analytics function then processes this data and the visualization function then displays it to the user. The entire system runs in the browser through Azures Static Web App service

Figure. System Architecture Diagram



B. Component Details

Binance API Data Retrieval

This component is responsible for contacting the Binance REST API and downloading the daily BTC candlestick data. It uses asynchronous JavaScript (with the use of async/await) to avoid blocking or stalling the webpage as when something is run asynchronously it is run in the background, also includes performance instrumentation to measure API latency. The module outputs two arrays: timestamps and closing prices.

Client Side Analytics

This module processes the price data and computes summary statistics including average price, highest price, and lowest price. It uses JavaScript array operations (reduce, Math.max/min) and updates the dashboard interface through Document Object Manipulation (DOM), which is basically just using JS to change the content/style/structure of a webpage while running. This component ensures that users receive quick analytical insights without requiring the use of a backend server.

Visualisation (Plotly.js)

This component generates a line chart of Bitcoin prices using Plotly.js. and the user is able to interact with this graph too. It dynamically updates based on the selected time window (30, 60, or 90 days) and includes code that I am able to track performance to record chart rendering time. The visualization is fully client side and requires no server-side rendering.

Azure Static Web Apps Hosting Layer

Azure Static Web Apps serves the application's HTML, CSS, JavaScript, and Plotly library files through a globally distributed Content Delivery Network. Because the system is serverless, Azure automatically handles the scaling, caching, and delivery of all content I implement. Deployment is automated through GitHub Actions, which rebuilds and publishes the site on after every commit (although it does take a second for this content to refresh on Azure).

C. Implementation Notes

The system is implemented entirely in JavaScript, using the Fetch API for the data retrieval aspect of my dashboard and Plotly.js is used for the visualization of the data gathered. The application is hosted on Microsoft Azure Static Web Apps, which provides an environment that is run serverless with automated CI/CD via GitHub Actions. No backend server, VM, or container is required, as any and all computation occurs in the browser itself. Key implementation challenges included the handling of asynchronous API calls, ensuring responsive chart rendering for larger datasets, and configuring Azure's deployment workflow to correctly serve my application from the /src directory as when I initially pushed up my dashboard and looked whether it worked or not I was met with Azures default webpage they include which meant that my filepath did not point towards my index.html at all.

Code Snippet:

```javascript
async function fetchBtcData(days) {
  const start = performance.now();  // start timer for performance measurement
  const url = "https://api.binance.com/api/v3/klines";
  const params = new URLSearchParams({
    symbol: "BTCUSDT",
    interval: "1d",
    limit: days.toString()
  });

  const response = await fetch(`${url}?${params.toString()}`);
  const data = await response.json();

  const end = performance.now();  // end timer for performance measurement

  console.log(`Data fetch and processing for ${days} days took ${end - start} ms`);

  const dates = [];
  const prices = [];

  for (const row of data) {
    const openTime = row[0];   // ms
    const close = parseFloat(row[4]);

    dates.push(new Date(openTime));
    prices.push(close);
  }


  return { dates, prices };
}
```

Async function (function that runs in the background/asynchronously) which fetches Bitcoin data from the Binance API, this function also includes the code I use to measure API latency for my performance evaluation

Measures:

- How long does it takes to contact the Binance API
- Binances response time
- How long it takes to download and parse the JSON file sent back from the Binance API
- All these measurements together return a value measured in milliseconds

```javascript
function updateStats(prices) {
  const avg =
    prices.reduce((sum, p) => sum + p, 0) / prices.length;
  const high = Math.max(...prices);
  const low = Math.min(...prices);

  document.getElementById("avg").textContent =
    `Average Price: $${avg.toFixed(2)}`;
  document.getElementById("high").textContent =
    `High Price: $${high.toFixed(2)}`;
  document.getElementById("low").textContent =
    `Low Price: $${low.toFixed(2)}`;
}
```

Computes my summary statistics (mean, max. min) with the BTC closing prices and updates my dashboard's interface:

- Reduce() is used to calculate my average
- Math.max/min() with my spread operator (…) is used to get my highest and lowest values

```javascript
function renderChart(dates, prices, days) {
  let title = "";

  if(days === 30) {
    title = "BTCUSDT Price - Last 30 Days";
  } else if(days === 60) {
    title = "BTCUSDT Price - Last 60 Days";
  } else if(days === 90) {
    title = "BTCUSDT Price - Last 90 Days";
  }

  const trace = {
    x: dates,
    y: prices,
    type: "scatter",
    mode: "lines",
    name: "BTCUSDT"
  };

  const layout = {
    title: title,
    xaxis: { title: "Date" },
    yaxis: { title: "Price (USDT)" }
  };
  const renderStart = performance.now();

  Plotly.newPlot("chart", [trace], layout, { responsive: true });

  const renderEnd = performance.now();

  console.log(`Chart render time (${days} days): ${renderEnd - renderStart} ms`);
}
```

This function is responsible for generating the interactive BTC chart displayed on my dashboard. It begins by selecting a chart title based on the number of days you have selected. Then using Plotly, I construct a trace object containing a date array and the corresponding closing prices.

This is then set as a line graph which is my main way of visualizing BTC/USDT price movements over time.

I also record a time stamp for performance measurements using performance.now() which is the exact same way ive done it in my fetchBtcData function:

- Sets chart tile based on the selected timeframe (30, 60, 90 days)
- Creates a Plotly line chart using the provided date and price arrays
- Defines the axis labels
- Measures client-side chart render time

```javascript
async function loadData(days) {
  try {
    const { dates, prices } = await fetchBtcData(days);
    updateStats(prices);
    renderChart(dates, prices, days);
  } catch (err) {
    console.error(err);
    alert("Failed to load data from Binance.");
  }
}
```

Another Async function that acts as my main controller for my dashboard. When selecting a time range, the loadData function is called and calls all the functions necessary for my data to be gathered and then represented on screen:

- Coordinates my entire data loading pipeline
- Waits for Binance API
- Updates my stats panel
- Renders my BTC price chart
- Provides error handling for failed API calls

```yaml
###### Repository/Build Configurations - These values can be configured to match your app requirements. ######
# For more information regarding Static Web App workflow configurations, please visit: https://aka.ms/swaworkflowconfig
app_location: "src" # App source code path
api_location: "" # Api source code path - optional
output_location: "src" # Built app content directory - optional
###### End of Repository/Build Configurations ######
```

By following the tutorial provided by Azure I was given a template github repository that included a yaml file to help with the automated CI/CD deployment to Azure:

- This specifies the folder that Azure should deploy (app_location: “src”)

IV. EXPERIMENT SETUP AND PERFORMANCE EVALUATION

A. Experiment / Deployment Setup

Parameter	Value / Specification	Notes
Cloud Provider / Platform	Microsoft Azure – Static Web Apps	Deployed using Azure for Students subscription via GitHub Actions (Continuous Integration and Continuous Delivery)
Instance / Node Type	Serverless Static Web Application	I didn't use a dedicated Virtual Machine, so Azure handles scaling and hosting automatically
CPU / vCPU	Not Applicable (serverless)	Azure Static Web Applications do not create a Virtual Machine for the site so Azure handles everything in the background
RAM	Not Applicable (Serverless)	The Application is stored in Azures Content Delicery Network (not from a machine with RAM I have access too or can control) so the Azure platform manages caching and memory usage automatically
Storage	Azure Static Web Application Storage (approximately less than 100mb out of Azures 250mb limit)	Only Static assets are stored (HTML, CSS, JavaScript and the Plotly Library)
OS / Environment	Azure Managed environment	Deployment was executed via Github actions taught to me via tutorials on Azure and to my understanding deploying the website and running it is done by Azure as well
Workload / Dataset	Binance BTC daily price data (between 30 and 60 days)	Fetched this data via Binance API (REST) directly
Benchmark Tool	I used Plotly.js (client side)	I used Plotly to generate an interactive line chart and put together the information scraped from the Binance website regarding Highest and Lowest daily prices on BTC

TABLE I. Experimental Environment Configuration.

B. Evaluation Metrics

So, in terms of evaluation metrics and whatnot, this is where this project gets a slight bit more challenging.

I measured the API latency and Chart rendering times as these were the metrics I believe directly reflect the behaviour of a cloud hosted, API driven web app. The API latency measures the time taken to contact the Binance REST API, receiving the JSON file, downloading it and parsing it. This is relevant as my dashboard relies entirely on external cloud data and therefore any delay in the API pipeline could absolutely affect user experience. With regards to chart rendering time it basically just measures how long it takes for Plotly.js to generate the interactive BTC price chart in the browser and helps to determine whether a large dataset would introduce increased overhead. This is important as the system in place is fully client side and I would need to know the impact of increasing the workload size.

C. Baseline / Comparison

The baseline for this is the 30 day dataset, which represents the smallest workload/minimum amount of data that must be processed. This baseline is compared against two larger workloads: 60 and 90 day datasets. These three scenarios should allow us to examine how the system behaves as the amount of data retrieved from the Binance API increases.

Because the application is fully client side and the Binance API returns relatively small JSON payloads (between 30 and 90 rows), the expectation is that performance differences will be minimal. Multiple measurements were collected for each workload, and natural fluctuations were observed due to network variability, API caching, and my serverless hosting. The comparison therefore focuses not on strict linear scaling, but on demonstrating that the system remains responsive and stable across all workload sizes.

The results show that in combination with my serverless architecture…CDN based hosting and my API of choice keeps load times and rendering faster than expected even when the amount of data im processing increases

D. Results

```
Data fetch and processing for 30 days took 259.5999999642372 ms        (index):68
Chart render time (30 days): 18.400000035762787 ms                     (index):129
Data fetch and processing for 60 days took 323.5 ms                    (index):68
Chart render time (60 days): 17.80000001192093 ms                      (index):129
Data fetch and processing for 90 days took 321.30000001192093 ms       (index):68
Chart render time (90 days): 15.600000023841858 ms                     (index):129
Data fetch and processing for 30 days took 325.2999999523163 ms        (index):68
Chart render time (30 days): 12.699999988079071 ms                     (index):129
Data fetch and processing for 60 days took 296.5999999642372 ms        (index):68
Chart render time (60 days): 16.599999964237213 ms                     (index):129
Data fetch and processing for 90 days took 342.5 ms                    (index):68
Chart render time (90 days): 11.900000035762787 ms                     (index):129
Data fetch and processing for 30 days took 259.80000001192093 ms       (index):68
Chart render time (30 days): 18.80000001192093 ms                      (index):129
Data fetch and processing for 60 days took 328.10000002384186 ms       (index):68
Chart render time (60 days): 15.899999976158142 ms                     (index):129
Data fetch and processing for 90 days took 290.10000002384186 ms       (index):68
Chart render time (90 days): 17.19999998807907 ms                      (index):129
```

Above I've added the console log printed by my application when loading into the webpage or changing the timeframe of the data you wish to look at. As we can see there isn't too much variation in the length of time it takes to gather and then display the information. There are definitely other factors that we need to consider when looking at these numbers and that will be discussed in my interpretation of my results

Days	*Average API Latency (milliseconds)*	*Average Render Time (milliseconds)*
30	*281*	*16*
60	*315*	*16*
90	*317*	*14*

V. DISCUSSION

A. Interpretation of Results

The results in Table II show that both API latency and chart rendering time remains at a constant across all three of my dataset sizes (30, 60, and 90 days). The average API latency ranges from 281 ms to 317 ms without too ridiculous of a fluctuation, while render times remain between 14 ms and 16 ms. These values indicate that the system's performance is quite reliable regardless of the selected timeframe. The small fluctuations seen between the workloads are expected in a cloud environment, where factors such as routing, API caching, and the users internet conditions in the moment can influence response times. Because of the Binance API returning relatively small JSON payloads, the difference between 30, 60, and 90 rows is fairly small, meaning the workload size does not impact the performance of my dashboard significantly. Overall, the results demonstrate that the dashboard remains responsive, with my total latency staying well below half a second across all scenarios.

B. Limitations

Although this works well there are a fair amount of limitations that I should consider and acknowledge. Firstly I rely on a single external API (Binance), this means that my performance us somewhat dependent on a third party service that is outside of my control.

Secondly my dataset size is quite small so my results wouldn't necessarily give a good representation of how the system would behave on larger workloads

Lastly since my application is wholly client side, performance could vary across devices with different CPU capabilities especially when it comes to low end mobile hardware for example.

C. Challenges Encountered

Initially my first challenge was figuring out how to actually deploy something to Azure as at this point in time I have not been able to complete the labs necessary for me to be super familiar with the platform. Coupled with me not completing two of the previous labs necessary for this deploying the application to Azure Static Web Apps also required learning how GitHub Actions automates CI/CD, as originally I ran into the issue of not giving the correct file paths to where my webpage was created so I had to ensure that the correct folder structure (src/) was used so the site would build and deploy successfully

One other challenge I encountered would be interpreting the variability in API my latency, which appeared inconsistent at first until I realised it was normal network behaviour rather than a flaw in the system.

VI. CONCLUSION

A. Summary

This project implemented a fully client side, serverless dashboard that retrieves Bitcoin pricing (candlestick) data from the Binance API, calculates summary statistics, and visualizes the results using Plotly.js. The system was deployed on Azure Static Web Apps using an automated CI/CD pipeline. Performance evaluation showed that API latency and chart rendering times remain consistently low across all tested workloads, demonstrating that the architecture is efficient and responsive for lightweight, API driven applications.

B. Recommendations for Future Work

- Widen the dashboard to accommodate and support additional cryptocurrencies or multiple APIs to compare performance across different providers.
- Incorporate real time data streaming or some form of WebSocket support to evaluate performance under continuous updates.
- Add more complex analytics (for example: moving averages, volatility indicators, anomaly/outlier detection) to explore how increased computation affects client side performance.
- Evaluate the system on mobile devices or lower powered hardware to assess performance across a wider range of system environments.

VII. SCREENCAST LINK

Cloud

. REFERENCES

[1] Mozilla Developer Network, "HTML: HyperText Markup Language," MDN Web Docs, Accessed: April 27th, 2026.

[2] Mozilla Developer Network, "CSS: Cascading Style Sheets," MDN Web Docs, Accessed: April 27th, 2026.

[3] Mozilla Developer Network, "JavaScript Guide," MDN Web Docs, Accessed: April 27th, 2026.

[4] Mozilla Developer Network, "Fetch API," MDN Web Docs, Accessed: April 27th, 2026.

[5] Binance, "Binance API Documentation," Binance Developers Portal, Accessed: April 27th, 2026.

[6] Plotly Technologies Inc., "Plotly.js Documentation," Plotly, Accessed:April 27th, 2026.

[7] Python Software Foundation, "Python Documentation," Python.org, Accessed: April 25th, 2026.

[8] Web Dev Simplified, "How To Use Fetch API (Learn Fetch API in 6 Minutes)," YouTube, 2020. Accessed: April 28th, 2026. [Online Video]. Available: https://youtu.be/MQdIiJh6AGc

[9] Fireship, "JavaScript Fetch API in 5 Minutes," YouTube, 2021. Accessed: April 27th, 2026. [Online Video]. Available: https://youtu.be/OEWQvjsA18A

[10] Traversy Media, "JavaScript Async/Await Crash Course," YouTube, 2017. Accessed: April 27th, 2026. [Online Video]. Available: https://youtu.be/cIStcPXjxKI

[11] Code With Ania Kubów, "How to Build a Crypto Dashboard Using JavaScript," YouTube, 2022. Accessed: April 26th, 2026. [Online Video]. Available: https://youtu.be/6ITp3egnJ54

[12] Binance Developers, "Algo Trading Quick Start Guide," Binance Developer Documentation. Accessed: April 26th, 2026. Available: https://developers.binance.com/docs/algo/quick-start

[13] Binance, "binance-connector-js: Official JavaScript Connector for Binance API," GitHub Repository. Accessed: April 26th, 2026. Available: https://github.com/binance/binance-connector-js

[14] AlgoTrading101, "Binance Python API Guide: How to Use the Binance API in Python," AlgoTrading101. Accessed: April 25th, 2026. Available: https://algotrading101.com/learn/binance-python-api-guide/

[15] W3Schools, "JavaScript Asynchronous Programming," W3Schools Online Web Tutorials. Accessed: April 26th, 2026. Available: https://www.w3schools.com/js/js_asynchronous.asp

[16] W3Schools, "JavaScript Tutorial," W3Schools Online Web Tutorials. Accessed: April 26th, 2026. Available: https://www.w3schools.com/js/default.asp

[17] W3Schools, "HTML Tutorial," W3Schools Online Web Tutorials. Accessed: April 26th, 2026. Available: https://www.w3schools.com/html/default.asp

[18] W3Schools, "CSS Tutorial," W3Schools Online Web Tutorials. Accessed: April 26th, 2026. Available: https://www.w3schools.com/css/default.asp

[19] W3Schools, "Node.js Crypto Module," W3Schools Online Web Tutorials. Accessed: April 27th, 2026. Available: https://www.w3schools.com/nodejs/nodejs_crypto.asp

[20] W3Schools, "Node.js REST API Tutorial," W3Schools Online Web Tutorials. Accessed: April 27th, 2026. Available: https://www.w3schools.com/nodejs/nodejs_rest_api.asp

[21] Microsoft, "Copilot for Web: AI-Assisted Development and Deployment Guidance," *Microsoft Documentation*, Accessed: April 29th, 2026. Available: https://copilot.microsoft.com

APPENDIX: Project Report Grading Overview (15 Marks Total)

Your report is graded holistically across the following criteria:

Criterion	Weight	Anticipated outcomes
Topic Relevance & Originality	~2 pts	Is the topic relevant to cloud data centres? Does it show original thinking or novel application?
Amount of Work	~2 pts	Is the scope commensurate with a module project? Does the work feel thorough and complete?
Technical Accuracy	~3 pts	Are technical explanations correct and precise? Are architecture, metrics, and methods described accurately?
System Architecture	~2 pts	Is the design well-explained and clearly illustrated? Are design choices justified?
Evaluation & Analysis	~3 pts	Are experiments/use cases well designed? Is evaluation thorough? Is analysis insightful?
Complexity of Techniques	~1 pt	How sophisticated are the methods used? Are advanced cloud/distributed techniques applied?
Writeup Quality	~2 pts	Is the report well-structured, clearly written, and properly formatted in IEEE style? Is it understandable by peers and general audience?

Please Note

- Please refer to SETU's Academic Integrity Policy for the use of AI/GenAI. AI tools can be used for assistance during project deployment (configuration, tutorials, error-assistance etc.) ONLY if they're properly referenced and documented within respective sections in this report. Please note that plagiarized/AI-generated reports ***shall not be graded***.
- The report will be graded on the structure and clarity (rather than the use of English language). Should you use any available tools (AI/non-AI) to improve your English language structure, grammar, and/or paraphrase, please ensure that you **cite** them appropriately in your report.
- **Late submissions** and **NO Submission** policies will be enforced for BOTH (project reports as well as the screencast).

Submission Checklist

- Include ALL code/configuration written or adapted by you (in the report or as a GitHub repo attachment).
- Do NOT include entire source code of off-the-shelf packages used without modification.
- Ensure Screencast link is **provided** within the report and is **accessible**.
- Submit via your GitHub repo before the **deadline** (**05:00 PM**, Friday **24 April 2026**).

Good luck!